MSML606 Extra Credit Project 2 Proposal

NutriMax: A Budget-Aware Grocery Optimizer

1. Team Composition

1. **Prathamesh Uravane**
2. **Sankeerth Billa**

2. Problem Statement

Grocery shopping on a fixed budget is a common real-world challenge, particularly for students, families, and individuals managing tight finances. The core question is: given a limited budget, which combination of grocery items maximizes overall nutritional value?

This is a direct instance of the 0/1 Knapsack Problem, a classic dynamic programming (DP) problem.

1. Each grocery item is either purchased (1) or not (0).
2. Items cannot be split or partially purchased.
3. The budget acts as the knapsack capacity, item prices are the weights, and nutritional scores are the values to maximize.

The application, NutriMax, will allow a user to input a grocery budget and receive an optimized shopping list that maximizes a composite nutrition score across calories, protein, fiber, and key vitamins all computed using the DP algorithm at its core.

3. Dataset

Source: USDA FoodData Central

URL: https://fdc.nal.usda.gov/download-datasets.html

The USDA FoodData Central database is a publicly available, government-maintained nutrition database. It provides detailed nutritional data for thousands of common food items in downloadable CSV format. Will use AI to extract useful fields from the repository.

Key Fields

- description → Food item name (e.g., "Whole Milk", "Chicken Breast")
- energy (kcal) →Caloric content per 100g serving
- protein (g) →Protein content per 100g serving
- fiber (g) → Dietary fiber per 100g serving
- vitamin_c, vitamin_a →Micronutrient content

- price_usd →Average US retail price (manually sourced from USDA ERS grocery price data and cross-referenced with public supermarket data)

4. Algorithm & Application Design

Algorithm: 0/1 Knapsack (Dynamic Programming)

The 0/1 Knapsack DP algorithm .The formulation is as follows:

- Input: List of N food items, each with a price and nutrition score; user-supplied budget B
- DP Table: dp[i][b] = maximum nutrition score achievable using the first i items with budget b
- Transition: For each item i with price p[i] and value v[i]:
- If p[i] <= b: dp[i][b] = max(dp[i-1][b], dp[i-1][b - p[i]] + v[i])
- Else: dp[i][b] = dp[i-1][b]
- Output: Traceback through the DP table to recover the selected items

5. AI Stetement

- AI not used at this stage. But for data extraction we will use AI.